



Project Proposal

Deploying a Lightweight RL Policy for Snake on micro:bit

Willie Liao – GEL8580

April 30, 2026

1 Introduction

Embedded AI is often discussed in terms of model accuracy, but in practice deployment constraints are just as important. On very small devices, the main question is often not whether a model can be trained, but whether a useful policy can be executed with limited memory and computation. Since I previously studied reinforcement learning in the last quarter, I want to use this project to explore how a learned policy can be applied in a much smaller embedded setting.

For this project, I plan to use the Snake game implemented on the micro:bit LED matrix. The micro:bit is a very constrained platform, so it is not realistic to run standard RL training on-device. Instead, I will study an inference-only setting: train a policy on a desktop machine, simplify its state and action representation, and then deploy the final policy to the micro:bit. The goal is to see whether a learned Snake agent can still be useful after being compressed enough to run on this type of hardware.

2 Project Goal

The main goal of this project is to study whether a lightweight RL policy can be deployed on a micro:bit and still perform reasonably on the Snake game. More specifically, I want to understand the tradeoff between policy quality and embedded feasibility.

This project is not trying to propose a new reinforcement learning algorithm. Instead, it focuses on deployment: how small the policy representation must be, how fast inference can run, and whether the learned policy is actually better than simpler baselines once it is placed on a constrained device.

3 Proposed Approach

I will build on the Snake game from the micro:bit lab. The game already provides the environment, including movement, growth, collision logic, and display on the 5x5 LED matrix. On top of this, I will define a compact state representation for control. Instead of using the full board as input, the agent will likely use a small set of features such as current heading, whether the front/left/right cells are dangerous, and the relative direction of the food.

Using this reduced state representation, I will first train a policy on a desktop machine. My initial plan is to use tabular Q-learning, since it naturally produces a small lookup-table policy that is easier to deploy on a microcontroller. At runtime, the micro:bit would only need to map the current game state to a compact feature vector and select an action from the learned table.

If time permits, I may also compare this lookup-table policy with a very small neural policy trained offline and executed on-device through a hand-written forward pass. This would allow a comparison between two policy representations: direct table lookup and small-network inference.

The deployed system will be inference-only. The micro:bit will not learn online. At each game step, it will read the current state, compute the compact features, choose an action, and update the snake accordingly.

4 Evaluation Plan

I plan to compare several agents:



- a random policy,
- a simple rule-based policy,
- an RL-based lookup-table policy,
- and, if feasible, a tiny neural policy.

These agents will be evaluated from two sides. The first is game performance. I will measure average score, average survival time, and common failure modes such as wall collisions and self-collisions. The second is embedded cost. I will measure policy size, approximate memory usage, and per-step decision latency on the micro:bit.

The main comparison I want to make is whether the RL policy gives enough benefit over a hand-designed rule-based policy to justify its added complexity. This is important because on a very small embedded platform, a slightly stronger policy may not be worthwhile if it costs much more memory or implementation effort.

5 Expected Outcome

I expect this project to produce a working Snake agent on micro:bit using inference only. The main result will likely not be a highly advanced RL system, but rather a practical study of what parts of reinforcement learning remain useful under severe hardware limits.

I also expect the comparison with rule-based and random baselines to make the result more meaningful. If the learned policy performs better while staying small enough to deploy, that would support the idea that even simple RL policies can be relevant in embedded AI. If the improvement is small, that would still be a useful finding, since it would show that for a platform like micro:bit, simpler methods may already be sufficient.

6 Scope and Risk

One risk is that the state representation may still be too large if it is not simplified enough. Another risk is that a neural policy may add implementation complexity without giving much improvement over a lookup table. To keep the project manageable, the minimum goal is to deploy one learned policy on the micro:bit and compare it with at least two non-learning baselines.

Overall, I think this project is a reasonable way to connect reinforcement learning from the previous quarter with the current course focus on embedded AI. It uses a familiar environment, but shifts the main question from training performance to deployment feasibility.

Note: I am also taking CE346 (Microcontroller Design) this quarter. Although this project uses the micro:bit platform, I do not plan to use the same work as my final project for CE346. The purpose here is to study lightweight RL inference as an embedded AI problem for this course.